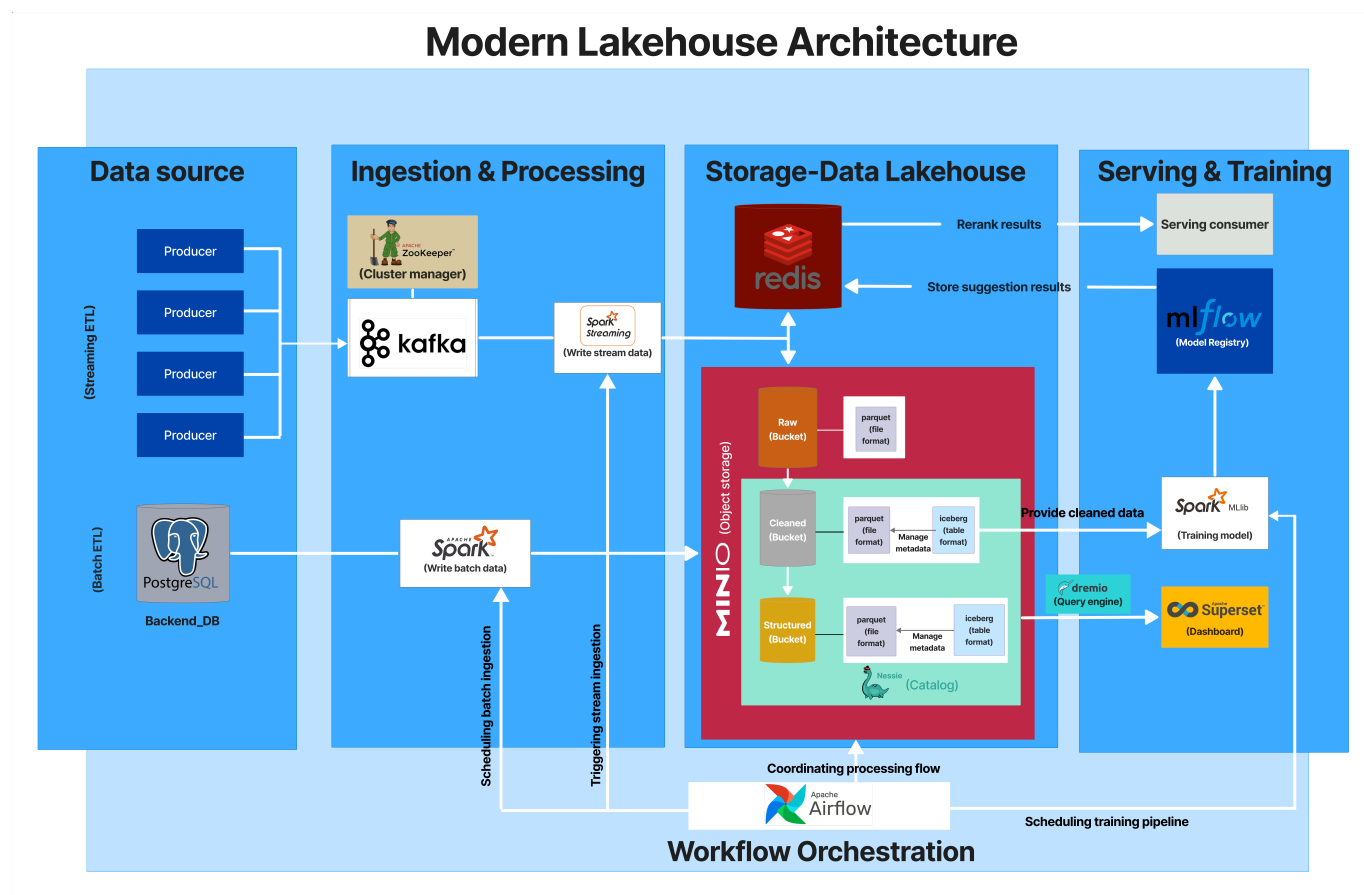


E-Commerce Lakehouse

This project implements a unified Lakehouse architecture for e-commerce analytics and real-time recommendation processing. The system integrates batch data processing and streaming event processing within the same platform, allowing historical analytics, recommendation model training, dashboard visualization, and real-time recommendation reranking to operate on shared Lakehouse data layers.



1. Project Overview

The project builds an end-to-end e-commerce Lakehouse system using open-source data engineering tools. Historical e-commerce data is loaded from PostgreSQL into a Lakehouse storage layer, processed through Bronze, Silver, and Gold stages, and used for business analytics and recommendation model training.

At the same time, real-time user behaviour events are streamed through Kafka and processed by Spark Structured Streaming. These events are used to update recommendation results by combining recent user behaviour with batch-trained ALS recommendation artifacts.

The final system supports two main outputs:

- An e-commerce analytics dashboard built with Dremio and Apache Superset.
- A real-time recommendation output generated from Kafka events, Redis cache, and ALS model artifacts.

2. Project Goals

The main goal of this project is to demonstrate how a single Lakehouse platform can support both batch and streaming workflows for e-commerce use cases.

The project focuses on:

- Building a unified Lakehouse architecture using MinIO, Apache Iceberg, and Project Nessie.
- Ingesting historical e-commerce data from PostgreSQL.
- Processing real-time user behaviour events from Kafka.
- Transforming raw data into cleaned and analytical datasets.
- Creating Gold-layer fact and dimension tables for dashboard analytics.
- Training an ALS collaborative filtering recommendation model using Spark MLlib.
- Tracking recommendation model metrics and artifacts with MLflow.
- Using streaming events to rerank recommendation results in near real time.
- Visualizing business metrics through Apache Superset.

3. System Architecture

3.1 Overall Architecture

The system contains two main data flows:

1. **Batch flow**

PostgreSQL stores historical e-commerce data such as users, products, orders, order items, categories, and reviews. Spark extracts these tables, stores raw data in the Bronze layer, cleans and standardizes data in the Silver layer, and creates analytical Gold-layer tables for dashboarding and recommendation training.

2. **Streaming flow**

Kafka receives user behaviour events such as page views and add-to-cart actions. Spark Structured Streaming consumes these events, combines them with trained ALS artifacts, uses Redis for recent behaviour caching, and publishes reranked recommendation results to a Kafka output topic.

The shared Lakehouse storage is built on MinIO, Apache Iceberg, and Project Nessie. Dremio is used to query Gold-layer Iceberg tables, while Superset is used to build the final analytics dashboard.

3.2 Architecture Components

Layer	Components	Purpose
Data Source	PostgreSQL, Kafka	Store historical data and stream user behaviour events
Ingestion	Apache Spark, Spark Structured Streaming	Load batch data and consume streaming events
Storage	MinIO	Store Bronze, Silver, Gold, checkpoint, and MLflow artifacts

Layer	Components	Purpose
Table Format	Apache Iceberg	Manage structured Lakehouse tables
Metadata Catalog	Project Nessie	Manage Iceberg catalog metadata and table versions
Processing	Apache Spark	Clean, transform, aggregate, and prepare data
Orchestration	Apache Airflow	Run batch pipeline tasks in order
Analytics	Dremio, Apache Superset	Query Gold tables and create dashboard visualizations
Recommendation	Spark MLlib ALS, MLflow, Redis	Train recommendation model, track metrics, and serve reranked results
Streaming Output	Kafka	Publish real-time recommendation results

4. Tech Stack

Category	Tools
Batch Source Database	PostgreSQL
Streaming Platform	Apache Kafka, ZooKeeper
Processing Engine	Apache Spark
Streaming Processing	Spark Structured Streaming
Object Storage	MinIO
Lakehouse Table Format	Apache Iceberg
Catalog and Versioning	Project Nessie
Workflow Orchestration	Apache Airflow
Query Engine	Dremio
Dashboard	Apache Superset
Recommendation Model	Spark MLlib ALS
Model Tracking	MLflow
Cache / Serving Layer	Redis
Deployment	Docker, Docker Compose

5. Repository Structure

```
.
├── airflow/
│   ├── dags/
│   │   ├── spark_batch_job.py
│   │   └── streaming_job.py
│   ├── logs/
│   ├── Dockerfile
│   └── requirements.txt
├── data/
│   ├── batch/
│   │   ├── categories.csv
│   │   ├── products.csv
│   │   ├── users.csv
│   │   ├── orders.csv
│   │   ├── order_items.csv
│   │   └── reviews.csv
│   └── stream/
│       ├── page_view.jsonl
│       ├── add_to_cart.jsonl
│       ├── purchase.jsonl
│       └── review.jsonl
├── image/
│   └── Lakehouse.png
├── scripts/
│   ├── database/
│   │   └── load_tables.py
│   ├── spark_jobs/
│   │   ├── bronze_batch_load.py
│   │   ├── silver_clean_transform.py
│   │   ├── gold_transform.py
│   │   ├── train_model.py
│   │   ├── streaming_flow.py
│   │   └── show_tables.py
│   └── streaming/
│       ├── event_producer.py
│       ├── test_consumer.py
│       └── test_consumer.py
├── batch_stimulate.py
├── stream_stimulate.py
├── docker-compose-lakehouse.yml
├── docker-compose-streaming.yml
├── .env
└── README.md
```

6. Data Sources

6.1 Batch Source Data

The batch source data is stored in PostgreSQL. It represents a transactional e-commerce database.

Main source tables:

Table	Description
<code>categories</code>	Product category information
<code>products</code>	Product details, category, brand, and price
<code>users</code>	User profile information
<code>orders</code>	Order-level transaction data
<code>order_items</code>	Product-level details for each order
<code>reviews</code>	Product ratings and review text from users

6.2 Streaming Event Data

The streaming source data represents real-time user behaviour events. These events are published to Kafka topics.

Main event types:

Event Type	Description
<code>page_view</code>	User views a product page
<code>add_to_cart</code>	User adds a product to cart
<code>purchase</code>	User purchases a product
<code>review</code>	User reviews a purchased product

Kafka topics used by the streaming flow:

```
events.page_view
events.add_to_cart
events.purchase
events.review
recommend.reranked
```

The topic `recommend.reranked` stores the final real-time recommendation output.

6.3 Synthetic Data Generation

This project uses synthetic e-commerce data instead of real customer data.

The batch data generation script creates:

- Categories
- Products
- Users
- Orders
- Order items
- Reviews

The streaming data generation script creates JSONL event files based on user purchase and review history. This makes the simulated streaming behaviour more connected to the historical batch dataset.

7. End-to-End Setup and Execution Guide

7.1 Prerequisites

Before running the project, make sure the following tools are installed:

- Docker
- Docker Compose
- Python 3.10+
- pip
- Git

Recommended local resources:

Resource	Recommended
RAM	12 GB or above
CPU	4 cores or above
Disk	20 GB free space or above

Required ports:

Service	Port
Airflow	8080
Superset	8088
MinIO API	9000
MinIO Console	9001
Dremio	9047
Dremio JDBC/ODBC	31010
Dremio Arrow Flight SQL	32010
Spark Master UI	9090

Service	Port
Spark Master	7077
MLflow	5001
PostgreSQL Airflow Metadata DB	5432
PostgreSQL Backend Source DB	5433
Kafka External Access	29092
Redis	6379
Redis Insight	8001
Nessie	19120

7.2 Configure Environment Variables

Create a `.env` file in the project root.

Example:

```
# PostgreSQL source database
POSTGRES_HOST=localhost
POSTGRES_PORT=5433
POSTGRES_DATABASE=ecommerce
POSTGRES_USER=ecommerce
POSTGRES_PASSWORD=ecommerce

# MinIO
MINIO_ENDPOINT=http://minio:9000
MINIO_ACCESS_KEY=minioadmin
MINIO_SECRET_KEY=minioadmin

# Kafka
KAFKA_BOOTSTRAP_SERVERS=kafka:9092

# Redis
REDIS_HOST=redis
REDIS_PORT=6379

# MLflow
MLFLOW_TRACKING_URI=http://mlflow_server:5000
MLFLOW_EXPERIMENT=als_recommend
MLFLOW_REGISTRY=als-model

# Superset
SUPERSET_SECRET_KEY=your_superset_secret_key

# Recommendation settings
```

```
CANDIDATE_POOL_SIZE=200  
FINAL_TOPK=10
```

For scripts that run directly from the host machine, PostgreSQL should use:

```
POSTGRES_HOST=localhost  
POSTGRES_PORT=5433
```

For services running inside Docker, the source PostgreSQL container is accessed internally as:

```
backend-postgres:5432
```

7.3 Start Lakehouse Infrastructure

Start the Lakehouse services:

```
docker compose -f docker-compose-lakehouse.yml up -d --build
```

Start the streaming services:

```
docker compose -f docker-compose-streaming.yml up -d
```

Check running containers:

```
docker ps
```

Expected main containers:

```
spark-master  
spark-worker  
postgres  
backend-postgres  
airflow-webserver  
airflow-scheduler  
minio  
nessie  
dremio  
superset  
mlflow_server  
zookeeper
```



```
kafka
redis
```

7.4 Verify Running Services

Open the following services in the browser:

Service	URL	Default Login
Airflow	http://localhost:8080	admin / admin
MinIO	http://localhost:9001	minioadmin / minioadmin
Dremio	http://localhost:9047	Create user on first login
Superset	http://localhost:8088	admin / admin
MLflow	http://localhost:5001	No login by default
Redis Insight	http://localhost:8001	No login by default
Spark Master UI	http://localhost:9090	No login by default

MinIO should contain the following buckets after startup:

```
bronze-layer
silver-layer
gold-layer
checkpoints
mlflow-models
```

7.5 Generate Source Data

If the CSV and JSONL files are already available in the [data/](#) folder, this step can be skipped.

Generate batch source data:

```
python batch_stimulate.py
```

Move generated CSV files into the batch data folder if needed:

```
mkdir -p data/batch

mv categories.csv data/batch/
mv products.csv data/batch/
mv users.csv data/batch/
mv orders.csv data/batch/
```

```
mv order_items.csv data/batch/  
mv reviews.csv data/batch/
```

Generate streaming event data:

```
python stream_stimulate.py
```

Expected streaming files:

```
data/stream/page_view.jsonl  
data/stream/add_to_cart.jsonl  
data/stream/purchase.jsonl  
data/stream/review.jsonl
```

7.6 Load Batch Source Data into PostgreSQL

Load CSV data into the PostgreSQL source database:

```
python scripts/database/load_tables.py
```

This script creates and loads the following tables:

```
categories  
products  
users  
orders  
order_items  
reviews
```

7.7 Run Batch Lakehouse Pipeline and Train Recommendation Model

Open Airflow:

```
http://localhost:8080
```

Login:

```
Username: admin
```

Password: admin

Go to:

Admin → Connections → + Add Connection

Create a new connection with the following values:

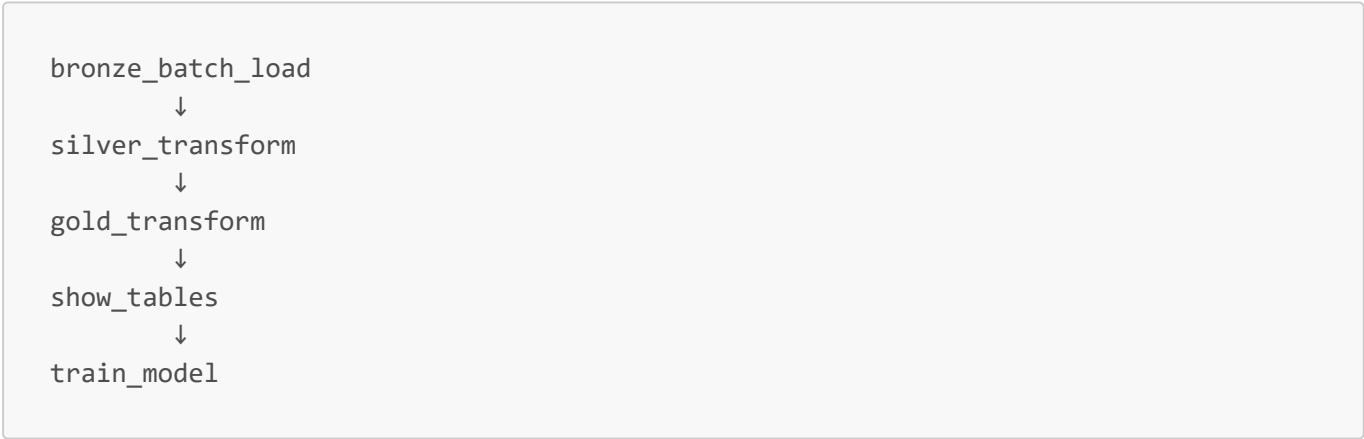
Connection Id = spark
Connection Type = Spark
Host = spark://spark-master
Port = 7077

Find the DAG:

batch-job

Trigger the DAG manually.

The batch pipeline should run the following stages:



The batch pipeline performs these tasks:

Stage	Description
Bronze ingestion	Extracts PostgreSQL tables and writes raw Parquet data to MinIO
Silver transformation	Cleans, validates, and standardizes raw data into Iceberg tables
Gold transformation	Creates analytical fact and dimension tables
ALS model training	Trains collaborative filtering model using <code>gold.fact_user_interactions</code>
Table validation	Prints or checks generated Lakehouse tables

After the DAG succeeds, the Lakehouse should contain:

```
bronze-layer/  
silver-layer/  
gold-layer/  
gold-layer/ml/als/user_factors  
gold-layer/ml/als/item_factors  
gold-layer/ml/als/user_mapping  
gold-layer/ml/als/item_mapping
```

7.8 Validate Lakehouse Tables

Use MinIO to confirm that data has been written to the Lakehouse layers:

```
http://localhost:9001
```

Check the following buckets:

Bucket	Expected Content
bronze-layer	Raw Parquet files from PostgreSQL ingestion
silver-layer	Cleaned Iceberg tables
gold-layer	Analytical Iceberg tables and ALS artifacts
mlflow-models	MLflow model artifacts

The main Gold-layer tables are:

```
gold.dim_users  
gold.dim_products  
gold.dim_date  
gold.fact_sales  
gold.fact_reviews  
gold.fact_user_interactions
```

7.9 Set Up Dremio Connection to the Lakehouse

Open Dremio:

```
http://localhost:9047
```

Create the first admin account if Dremio asks for initial setup.

Add a new source:

```
Source type: Nessie
Source name: lakehouse
```

Use the following Nessie configuration:

```
Nessie endpoint: http://nessie:19120/api/v2
Authentication: None
Reference: main
```

Configure MinIO / S3 access:

```
Access Key: minioadmin
Secret Key: minioadmin
Root Path: /
```

Use the following connection properties if Dremio requires them:

```
fs.s3a.endpoint = minio:9000
fs.s3a.path.style.access = true
dremio.s3.compat = true
fs.s3a.connection.ssl.enabled = false
```

After saving the source, verify that Dremio can access the Iceberg tables created by Spark.

Expected tables include:

```
dim_users
dim_products
dim_date
fact_sales
fact_reviews
fact_user_interactions
order_items
orders
products
reviews
users
```

7.10 Set Up Superset Connection to Dremio

Open Superset:

```
http://localhost:8088
```

Login:

```
Username: admin  
Password: admin
```

Go to:

```
Settings → Database Connections → + Database
```

Choose Dremio or SQLAlchemy URI connection.

Use this connection format:

```
dremio+flight://<DREMIO_USERNAME>:<DREMIO_PASSWORD>@dremio:32010/?  
UseEncryption=false
```

Example:

```
dremio+flight://admin:admin@dremio:32010/?UseEncryption=false
```

Test the connection. If the test succeeds, save the database connection.

Then create Superset datasets from the Dremio Gold-layer tables:

```
dim_users  
dim_products  
dim_date  
fact_sales  
fact_reviews  
fact_user_interactions
```

7.11 Create Analytics Dashboard in Superset

In Superset, create charts from the Gold-layer datasets.

Recommended dashboard name:

E-Commerce Analytics Dashboard

Recommended charts:

Chart	Source Tables	Metric / Purpose
Total Revenue KPI	fact_sales	Sum of item_total
Total Orders KPI	fact_sales	Count distinct order_id
Total Users KPI	dim_users	Count users
Monthly Sales Trend	fact_sales, dim_date	Revenue by month
Monthly Orders Trend	fact_sales, dim_date	Orders by month
Top Products by Revenue	fact_sales, dim_products	Product revenue ranking
Top Product Categories	fact_sales, dim_products	Revenue by category
Review Rating Distribution	fact_reviews	Count reviews by rating
Average Product Rating	fact_reviews, dim_products	Average rating by product

Dashboard creation flow:



7.12 Check MLflow Model Metrics

Open MLflow:

http://localhost:5001

Open the experiment:

```
als_recommend
```

Check the latest run.

Expected logged metrics:

```
rmse  
precision_at_10  
recall_at_10
```

Expected artifacts:

```
als_model  
indexer/user_labels.json  
indexer/item_labels.json
```

The training job also stores ALS serving artifacts in MinIO:

```
s3a://gold-layer/ml/als/user_factors  
s3a://gold-layer/ml/als/item_factors  
s3a://gold-layer/ml/als/user_mapping  
s3a://gold-layer/ml/als/item_mapping
```

These artifacts are used later by the real-time recommendation pipeline.

7.13 Start Streaming Event Producers

Start Kafka and Redis first if they are not already running:

```
docker compose -f docker-compose-streaming.yml up -d
```

Run the event producer from the host machine:

```
python scripts/streaming/event_producer.py
```

The producer sends events to Kafka topics:

```
events.page_view  
events.add_to_cart
```



```
events.purchase  
events.review
```

The producer continuously sends random user behaviour events from the generated JSONL files.

7.14 Run Real-Time Recommendation Pipeline

Open Airflow:

```
http://localhost:8088
```

Find the streaming DAG:

```
streaming-job
```

The streaming DAG should run the following Spark job:

```
scripts/spark_jobs/streaming_flow.py
```

The streaming job performs the following operations:

```
Consume Kafka events  
    ↓  
Parse and validate event JSON  
    ↓  
Load ALS user factors and item factors  
    ↓  
Generate candidate recommendations  
    ↓  
Use Redis recent behaviour for reranking  
    ↓  
Write final top-K recommendations to Redis  
    ↓  
Publish result to Kafka topic recommend.reranked
```

Main output topic:

```
recommend.reranked
```

7.15 Check Real-Time Recommendation Output

Run the recommendation consumer:

```
python scripts/streaming/test_consumer.py
```

Expected output format:

```
{
  "user_id": "123",
  "recommendations": ["145", "872", "91", "300", "44"],
  "source": "als_realtime",
  "metrics": {
    "batch_id": 10,
    "input_rows": 50,
    "emitted_users": 12,
    "latency_seconds": 1.23,
    "throughput_events_per_sec": 40.65
  }
}
```

This confirms that the streaming recommendation pipeline is working.

8. System Interfaces

Service	URL	Purpose	Login
Airflow	http://localhost:8080	Run and monitor batch pipeline	admin / admin
MinIO Console	http://localhost:9001	Inspect Lakehouse buckets	minioadmin / minioadmin
Dremio	http://localhost:9047	Query Iceberg tables	Create account on first login
Superset	http://localhost:8088	Build and view dashboard	admin / admin
MLflow	http://localhost:5001	View model runs and metrics	No login by default
Redis Insight	http://localhost:8001	Inspect Redis cache	No login by default
Spark Master UI	http://localhost:9090	Monitor Spark cluster	No login by default
Nessie API	http://localhost:19120	Iceberg catalog metadata API	No login by default
Kafka	localhost:29092	External Kafka access from host	No login by default